

Assignment 1 AnwSys: Enterprise Programming

Gregor Arlt

Projektidee und Erfüllung der Mindestkomplexität

Die Anwendung ist ein interaktiver, terminalbasierter To-do-Manager. Ursprünglich an das textbasierte Referenzprojekt „todo_txt“ angelehnt, wurde die Architektur zugunsten der Komplexität und Fehleranfälligkeit auf eine SQLite-Datenbank umgestellt. Dies verhindert Probleme beim Parsen von Text-Strings und ermöglicht elegante, sichere Datenmanipulationen mit SQL (unter Nutzung der `sqlite-jdbc`-Abhängigkeit).

Der Produktionscode gliedert sich in vier Kernklassen:

- **Main.java:** Einstiegspunkt der Anwendung inklusive Hauptschleife und Menüführung.
- **Methods.java:** Agiert als Vermittler, verarbeitet User-Eingaben, sammelt Parameter und delegiert diese an die Datenbanklogik.
- **DatabaseMethods.java:** Enthält die Methoden der SQL-Datenbankoperationen
- **Task.java:** Repräsentiert das Datenmodell einer Aufgabe und demonstriert den Einsatz von Lombok zur Codereduktion.

Die sechs geforderten Anforderungen wurden durch das Zusammenspiel der Klassen `Methods` und `DatabaseMethods` wie folgt umgesetzt:

- **1. Alle & 2. Offene To-dos ausgeben:** `Methods.listAll()` und `listOpen()` rufen die entsprechenden Methoden `readValue()` bzw. `readOpen()` auf. Die Daten werden über `SELECT *-Abfragen` (bei offenen gefiltert durch `WHERE checked = 0`) geladen und als formatierte Tabelle im Terminal präsentiert.
- **3. Suchen:** `Methods.search()` leitet die Parameter an `searchTable()` weiter. Der SQL-Operator `LIKE` ermöglicht dabei das zuverlässige Finden von Teilbegriffen (z. B. „Ler“ für „Lernen“).
- **4. Als erledigt markieren:** Nach Auswahl einer ID über `Methods.done()` ändert `updateTable()` die `checked`-Spalte des Datensatzes per `UPDATE`-Befehl auf 1.
- **5. Mehrere suchen & ändern:** `Methods.searchAndUpdate()` verlangt ein Suchkriterium sowie eine Zielspalte. `severalUpdatesTable()` überschreibt diese Spalte daraufhin bei allen gefundenen SQL-Treffern.
- **6. Neues To-do anlegen:** `Methods.add()` fragt alle Details ab und bündelt sie in einem neuen `Task`-Objekt. Dessen Attribute werden über `insertValues()` als neuer Datensatz in der Datenbank gespeichert.

Benutzeroberfläche und variabler Programmfluss

Zur Erfüllung der Anforderungen wurde eine textbasierte Terminalanwendung (CLI) implementiert. Der vom Benutzer gesteuerte, dynamische Programmfluss wird durch ein Hauptmenü realisiert, zu dem das Programm nach Beendigung einer ausgewählten Operation automatisch zurückkehrt.

Die kontinuierliche Ausführung und die wiederholte Menüanzeige werden in `Main.java` über eine `while`-Schleife gesteuert. Diese läuft, solange die boolesche Variable `running` den Wert `true` besitzt. Benutzereingaben werden zeilenweise via `java.util.Scanner` erfasst und innerhalb eines `try-catch`-Blocks mittels `Integer.parseInt()` verarbeitet, um Laufzeitfehler durch fehlerhafte Eingaben (z. B. Buchstaben) zu verhindern.

```
1 boolean running = true;
2 Scanner scan = new Scanner(System.in);
3
4 while(running == true) {
5
6     System.out.println("%%%%%%%%%%%%%% Menu %%%%%%%%%%%%%%");
7     Methods.menu(); //Ausgelagertes printing des Menues
8     System.out.println("-----");
9     System.out.println("Enter number of desired operation: ");
10
11     try {
12         int cmd = Integer.parseInt(scan.nextLine());
13
14         //switch-case-Block ...
15
16     } catch (RuntimeException e) {
17         System.out.println("The entered command does not exist.
18         Did you enter a whole number for the operation?");
19     }
20 }
```

Listing 1: While-Schleife in Main.java

Die Menü-Aausgabe ist in die Klasse `Methods.java` ausgelagert worden. Die statische Methode `menu()` wird bei jedem Schleifendurchlauf aufgerufen und gibt die verfügbaren Befehle im Terminal aus.

```
1 public static void menu(){
2     System.out.println("1 - add");
3     System.out.println("2 - list all");
4     System.out.println("3 - list all open tasks");
5     System.out.println("4 - search");
6     System.out.println("5 - mark as done");
7     System.out.println("6 - search and update");
8     System.out.println("7 - help");
9     System.out.println("8 - exit");
10 }
```

Listing 2: Darstellung des Hauptmenüs in Methods.java

Die Verzweigung des Programmflusses erfolgt über einen `switch-case`-Block innerhalb der Schleife. Je nach eingegebener Ganzzahl (`cmd`) wird die entsprechende statische Methode aus `Methods.java` aufgerufen und

das Datenbankverbindungsobjekt `conn1` übergeben. Bei Auswahl von Option 8 wird `running` auf `false` gesetzt, wodurch das Programm terminiert. Ganze Zahlen, welche nicht als Operation definiert sind (Bsp. 9 oder -5 etc.) werden über den `default`-Zweig abgefangen.

```
1 switch (cmd) {
2     case 1 -> { Methods.add(conn1); }
3     case 2 -> { Methods.listAll(conn1); }
4     case 3 -> { Methods.listOpen(conn1); }
5     case 4 -> { Methods.search(conn1); }
6     case 5 -> { Methods.done(conn1); }
7     case 6 -> { Methods.searchAndUpdate(conn1); }
8     case 7 -> { Methods.help(); }
9     case 8 -> { running = false; }
10    default -> {
11        System.out.println("Unknown command! Type in 'help' for further
12        instructions");
13    }
```

Listing 3: Programmfluss-Steuerung mittels Switch-Case in Main.java

Abfrage und Speicherung von User-Input

Ein wesentlicher Bestandteil der Anwendung ist die Erfassung von Input-Daten, welche den weiteren Verlauf und Zustand des Programms beeinflussen. In nahezu jeder Funktionalität der Anwendung wird eine `java.util.Scanner`-Instanz genutzt, um Daten vom Benutzer abzufragen. Diese Eingaben legen zum Beispiel fest, welche Operation ausgewählt werden soll oder welche exakten Werte in die Datenbank eingefügt oder aus ihr abgerufen werden müssen. Dadurch werden spätere Such- oder Ausgabeoperationen direkt durch vorherigen Eingaben beeinflusst.

Aus Platzgründen wird das anhand der Methode `Methods.add()` zur Anlage einer neuen Aufgabe demonstriert. Der User wird hierbei interaktiv und schrittweise durch die Datenerfassung geführt.

Nach der verpflichtenden Eingabe des Aufgabennamens ("1 - add) wird ein neues `Task`-Objekt instanziiert. Die Abfrage der Attribute (Name, Priorität, Tags etc.) erfolgt sequenziell. Dem User wird dabei die Möglichkeit gegeben, Eingaben durch das Drücken der Enter-Taste zu überspringen.

Die Zuweisung der erfassten Werte an das Datenobjekt erfolgt über die jeweiligen Setter-Methoden. Diese vermeidbare Zuweisung wurde an dieser Stelle bewusst implementiert, um die korrekte Funktionsweise und Integration der durch *Lombok* automatisch generierten Setter-Methoden in der `Task`-Klasse zu prüfen.

```
1 public static void add(Connection conn) {
2     //... logging
3
4     Scanner scan = new Scanner(System.in);
5     System.out.println("Add the name of the task:");
6     String name = scan.nextLine();
7
8     Task task = new Task(0, name);
9     //Eine frisch erstellte Aufgabe ist nicht erledigt -> 0
10
11    System.out.println("Add priority?: (Press Enter to skip)");
12    String prio = scan.nextLine();
```

```

13     task.setPriority(prio);
14
15     System.out.println("Add a due date?: (Press Enter to skip)");
16     String due = scan.nextLine();
17     task.setDue_date(due);
18
19     // weitere Abfragen mit selber Logik (Platz sparen)
20
21     DatabaseMethods.insertValues(conn, task.getChecked(), task.getTask(),
22                                 task.getPriority(), task.getDue_date(),
23                                 task.getProject_tag(), task.getContext_tag(),
24                                 task.getSpeacial_tag());
25 }

```

Listing 4: Interaktive Datenerfassung und Objekterzeugung in Methods.add()

Nachdem das Task-Objekt vollständig mit den Benutzereingaben befüllt wurde, werden die Daten aus dem Objekt über die entsprechenden Getter-Methoden ausgelesen (Erneut Lombok prüfen) und an die Methode `DatabaseMethods.insertValues()` übergeben. Diese übernimmt die Speicherung, indem sie die gesammelten Parameter in ein SQL-Statement übersetzt und in die Datenbank einpflegt. Ab diesem Moment beeinflussen die eingegebenen Daten den Programmfluss, da sie bei nachfolgenden Operationen (wie z. B. `list all` oder `search`) von der Anwendung ausgelesen und verarbeitet werden.

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Menu %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
1 - add
2 - list all
3 - list all open tasks
4 - search
5 - mark as done
6 - search and update
7 - help
8 - exit
-----
Enter number of desired operation:
1
Add the name of the task:
SQL lernen
Add priority?: (Press Enter to skip)
A
Add a due date?: (Press Enter to skip)
26.06.26
Add a project tag?: (Press Enter to skip)
|

```

Abbildung 1: Abfrage des Users

```

Enter number of desired operation:
2
Listing all tasks:

```

ID	Done	Task	Priority	Due	Project	Context	Individual
1	[X]	grger		20.05.2025			
2	[X]	Wischen	A	26.02.26	Uni	Uni	Dringend
3	[X]	Hausaufgabe	A	23.05.2026	Uni	AnwSys	Dringend
4	[X]	Duschen	B	12.06.2026	Pflege		
5	[X]	Hausaufgabe 4 bearbeiten (Tut2)	A	26.03.25	Uni	Uni	
6	[X]	Hausaufgabe InfTech II abgeben	A	26.02.2024	Uni	InfTechII	Dringende Aufgaben, bald Abgabe
7	[X]	TUT7			InfTechII	Uni	Dringende Aufgaben, bald Abgabe
8	[]	Staubsaugab			Geändert123456		Alle Aufgaben mit Staub
9	[]	Test			Geändert123456		
10	[]	null		asd	a		
11	[X]						
12	[]	a	a	a	a	a	a
13	[]	1	2	3	4	5	6
14	[]	SQL lernen	A	26.06.26	Uni		Dringend

Abbildung 2: Speicherung der Aufgabe in der Datenbank

Nutzung von Maven

Für das Build-Management und das Automatisieren von Abhängigkeiten wird Maven verwendet. Die benötigten Bibliotheken sind in der `pom.xml` wie folgt definiert:

- `sqlite-jdbc`: Stellt den JDBC-Treiber für die Anbindung und Interaktion mit der SQLite-Datenbank zur Verfügung

```
1 <dependency>
2   <groupId>org.xerial</groupId>
3   <artifactId>sqlite-jdbc</artifactId>
4   <version>3.53.1.0</version>
5   <scope>compile</scope>
6 </dependency>
```

Listing 5: SQLite JDBC Dependency

- `lombok`: Reduziert Boilerplate-Code durch die automatische Generierung mittels Annotationen

```
1 <dependency>
2   <groupId>org.projectlombok</groupId>
3   <artifactId>lombok</artifactId>
4   <version>1.18.46</version>
5   <scope>compile</scope>
6 </dependency>
```

Listing 6: Lombok Dependency

- `log4j-core` / `log4j-api`: Dient als Logging-Framework, wobei die Schnittstelle (API) und die Implementierung (Core) getrennt eingebunden werden.

```
1 <dependency>
2   <groupId>org.apache.logging.log4j</groupId>
3   <artifactId>log4j-api</artifactId>
4   <version>2.26.0</version>
5   <scope>compile</scope>
6 </dependency>
7
8 <dependency>
9   <groupId>org.apache.logging.log4j</groupId>
10  <artifactId>log4j-core</artifactId>
11  <version>2.26.0</version>
12  <scope>compile</scope>
13 </dependency>
```

Listing 7: Log4J Dependencies (API & Core)

- `JUnit`: Wird für die Implementierung und Ausführung von Unit-Tests verwendet.

```
1 <dependency>
2   <groupId>org.junit.jupiter</groupId>
3   <artifactId>junit-jupiter-api</artifactId>
```

```

4     <version>6.1.0</version>
5     <scope>test</scope>
6 </dependency>

```

Listing 8: JUnit Jupiter API Dependency

Die verwendeten Bibliotheken wurden aus dem zentralen Maven-Repository (<https://mvnrepository.com>) bezogen. Hierbei wurde stets die aktuellste stabile Version gewählt, um bekannte Fehler zu vermeiden und eine hohe Stabilität zu gewährleisten.

Im Kopfbereich der pom.xml wurden Metadaten definiert, die das Projekt spezifizieren. Durch die Integration der Tags <name>, <description> und <developers> ist eine Zuordnung des Anwendungszwecks sowie der Urheberschaft gewährleistet.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.
    org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6
7     <groupId>anwsys.ai</groupId>
8     <artifactId>todo-manager</artifactId>
9     <version>1.0.0</version>
10
11     <name>To-Do-Manager</name>
12     <description>Ein CLI-basierter To-Do-Manager, welcher mit einer SQLite-
    Datenbank interagiert</description>
13     <developers>
14         <developer>
15             <id>g.arlt</id>
16             <name>Gregor Arlt</name>
17         </developer>
18     </developers>
19
20     <properties>
21         <maven.compiler.source>21</maven.compiler.source>
22         <maven.compiler.target>21</maven.compiler.target>
23         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
24     </properties>
25 </project>

```

Listing 9: Projekt-Metadaten in der pom.xml

Nutzung von JUnit für Unit-Tests (Standard und Grenzfälle)

Zur Absicherung der Funktionalität der Klasse `DatabaseMethods` wird das Framework JUnit 5 eingesetzt. Die implementierten Unit-Tests überprüfen dabei die einzelnen Datenbankmethoden sowohl hinsichtlich ihrer korrekten Funktion bei regulären Eingaben als auch hinsichtlich ihres Verhaltens bei Grenz- und Fehlerfällen. Dadurch wird sichergestellt, dass die Methoden unter unterschiedlichen Bedingungen zuverlässig arbeiten.

Um eine vollständige Trennung von Test- und Produktivdaten zu gewährleisten, verwendet die Testklas-

se `DatabaseTests` eine temporäre In-Memory-Datenbank (`jdbc:sqlite::memory:`). Mithilfe der JUnit-Annotationen `@BeforeEach` und `@AfterEach` wird vor jeder Testausführung eine neue Datenbankinstanz erzeugt und nach Abschluss des Tests wieder geschlossen. Dadurch entstehen reproduzierbare Testergebnisse, da jeder Test mit einem definierten Ausgangszustand beginnt.

Die verschiedenen Methoden der Klasse `DatabaseMethods` werden jeweils durch eigene Unit-Tests überprüft. Eine Testung der anderen Klassen ist irrelevant, da diese größtenteils aus der Abfrage von User-Input bestehen. Aus Platzgründen wird die Vorgehensweise jedoch exemplarisch anhand der Methode `insertValues()` erläutert. Im Test `insertValuesTest()` wird zunächst ein Datensatz mit typischen Eingabewerten in die Testdatenbank eingefügt. Anschließend wird mithilfe einer SQL-Abfrage geprüft, ob der Datensatz tatsächlich gespeichert wurde und ob alle übergebenen Werte korrekt in den jeweiligen Datenbankspalten abgelegt wurden. Auf diese Weise kann sichergestellt werden, dass die Methode bei regulären Eingaben erwartungsgemäß funktioniert. Nach demselben Prinzip werden auch die weiteren Methoden der Klasse getestet.

```
1 @Test
2 void insertValuesTest() throws SQLException {
3     DatabaseMethods.insertValues(testConnection, 0, "Tests erledigen", "A",
4         "26.06.2026", "Assignments", "Uni", "Dringend");
5
6     PreparedStatement pstmt = testConnection.prepareStatement(
7         "SELECT * FROM ToDo WHERE taskName = ?");
8
9     pstmt.setString(1, "Tests erledigen");
10    ResultSet rs = pstmt.executeQuery();
11
12    assertTrue(rs.next());
13    assertEquals("Tests erledigen", rs.getString("taskName"));
14 }
```

Listing 10: Auszug des Unit-Tests für `insertValues()`

Neben Standardfällen werden gezielt Grenzfälle berücksichtigt. Ein Beispiel hierfür ist der Test `insertValuesTestNull()`, der überprüft, ob die Methode einen ungültigen Datensatz mit einem fehlenden Aufgabennamen korrekt zurückweist. Da der Aufgabenname ein Pflichtfeld darstellt, muss in diesem Fall eine `IllegalArgumentException` ausgelöst werden. Durch diesen Test wird sichergestellt, dass fehlerhafte Eingaben bereits vor dem eigentlichen Datenbankzugriff erkannt und abgefangen werden.

```
1 @Test
2 void insertValuesTestNull() {
3     assertThrows(IllegalArgumentException.class, () -> {
4         DatabaseMethods.insertValues(testConnection, 0, null,
5             "", "", "", "", "");
6         "Sollte IllegalArgumentException werfen, weil Name null ist");
7     });
8 }
```

Listing 11: Grenzfällttest für einen fehlenden Aufgabennamen

Zur Erhöhung der Testabdeckung werden zusätzlich Fehlerfälle auf Datenbankebene simuliert. Hierzu wird in speziellen Tests die Datenbankverbindung vor dem Methodenaufruf bewusst geschlossen. Dadurch wird eine `SQLException` provoziert und überprüft, ob die jeweilige Methode den Fehler intern behandelt, ohne die

Element ^	Class, %	Method, %	Line, %	Branch, %
▼ org.example	33% (1/3)	42% (9/21)	48% (138/285)	42% (22/52)
DatabaseMethods	100% (1/1)	100% (9/9)	97% (138/141)	84% (22/26)
Main	0% (0/1)	0% (0/2)	0% (0/26)	0% (0/10)
Methods	0% (0/1)	0% (0/10)	0% (0/118)	0% (0/16)
Task	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)

Abbildung 3: 97% Line-coverage bei DatabaseMethods.java

Anwendung abstürzen zu lassen. Mit `assertDoesNotThrow()` wird dabei verifiziert, dass die Exception ordnungsgemäß abgefangen wird. Auf diese Weise werden auch die Fehlerbehandlungsroutinen der Anwendung getestet und die Coverage der Testklasse erhöht.

```

1
2 void creatingTableTestSQLException() throws SQLException {
3     testConnection.close();
4     assertDoesNotThrow(() -> DatabaseMethods.createTable(
5         testConnection), "Sollte Exception fangen und ausgeben");
6 }

```

Listing 12: Exemplarisch: creatingTableTestSQLException()

Nutzung von Lombok

Lombok dient der automatischen Generierung von Boilerplate-Code zur Kompilierzeit. In diesem Projekt kam die Bibliothek für drei primäre Anwendungsfälle zum Einsatz. Die Umsetzung erfolgt dabei vorwiegend durch das Setzen von Annotationen auf Klassenebene. Zunächst findet sich in der Klasse `Task.java` ein klassischer Anwendungsfall von Lombok. Dort werden Getter, Setter und Konstruktoren automatisiert durch die Annotation `@Data` generiert. Zudem wurden Felder, die keine Nullwerte zulassen dürfen, mit `NonNull` annotiert.

```

1 @Data
2 public class Task {
3     @NonNull
4     private int checked;
5     //...
6     @NonNull
7     private String task;
8     //...
9 }

```

Listing 13: Auszug aus Task.java

Weiterhin wurden die Klassen `Methods.java` und `DatabaseMethods.java` mit der Annotation `@UtilityClass` versehen. Diese weist Lombok an, die Struktur der Klasse entsprechend einer klassischen Hilfsklasse anzupassen. Konkret bedeutet dies, dass die Klasse implizit als `final` deklariert und mit einem privaten Konstruktor versehen wird, um sowohl eine Vererbung als auch eine versehentliche Instanziierung auszuschließen. Gleichzeitig werden alle darin enthaltenen Felder und Methoden automatisch als `static` kompiliert.

```

1 @UtilityClass

```

```

2 public class Methods {
3     //...
4 }
5
6 @UtilityClass
7 public class DatabaseMethods {
8     //...
9 }

```

Listing 14: Auszug der Utility-Klassen

Zuletzt wurde für das Logging mit Log4J2 in den Klassen `Main.java`, `Methods.java` und `DatabaseMethods.java` die Annotation `@Log4j2` verwendet. Dies erspart die manuelle Erstellung eines Logger-Objekts und ermöglicht direkt den Aufruf entsprechender Logging-Methoden wie beispielsweise `log.info()`.

```

1 @Log4j2
2 public class Main {
3     //...
4 }
5
6 @Log4j2
7 public class Methods {
8     //...
9 }
10
11 @Log4j2
12 public class DatabaseMethods {
13     //...
14 }

```

Listing 15: Auszug der Klassen mit Logging-Annotation

Nutzung von Log4J2 für das Logging

Im Projekt wird zwischen der User-Kommunikation (`System.out.println`) und dem entwicklergerichteten Logging zum Debugging getrennt. Aufgrund bekannter Sicherheitslücken im veralteten Log4j kommt hierfür das moderne Framework Log4j2 zum Einsatz. (siehe Section Lombok)

Dessen Einstellung erfolgt über die Konfigurationsdatei `log4j2.xml` im Verzeichnis `src/main/resources`. Der `<Console>`-Appender leitet die Logs an die Standardausgabe (`SYSTEM_OUT`) weiter. Das `<PatternLayout>` definiert dabei ein einheitliches Format (`%d{...} [%t] %-5level ...`), das jeden Eintrag mit Zeitstempel, ausführendem Thread, Loglevel, Ursprungsklasse und Nachricht versieht. Über den `<Root>`-Logger wird das Basis-Loglevel auf `info` gesetzt. Dadurch werden irrelevante Debug-Meldungen im regulären Betrieb gefiltert und die Konsole bleibt übersichtlich.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <Configuration status="WARN">
3     <Appenders>
4         <Console name="Console" target="SYSTEM_OUT">
5             <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level

```

```

6         %logger{36} - %msg%n"/>
7     </Console>
8 </Appenders>
9 <Loggers>
10     <Root level="info">
11         <AppenderRef ref="Console"/>
12     </Root>
13 </Loggers>
14 </Configuration>

```

Listing 16: Auszug aus Log4j2.xml

Dieses Logging-Konzept wird projektweit konsistent angewendet, wie die Methode `DatabaseMethods.updateTable()` exemplarisch zeigt:

```

1 public static void updateTable(Connection conn, String targetColumn,
2     String
3     searchColumn, String filterColumn, String filter) {
4     log.debug("DatabaseMethods.updateTable() aufgerufen.
5     Setze '{}' = '{}' wo '{}' = '{}'", targetColumn, searchColumn,
6     filterColumn, filter);
7     //...
8     try {
9         //...Logikausfuehrung...
10        log.info("Update erfolgreich");
11
12    } catch (SQLException e) {
13        log.error("Update fehlgeschlagen.
14        Konnte Spalte '{}' nicht auf Wert '{}' (Filter: {}={}) setzen.",
15        targetColumn, searchColumn, filterColumn, filter, e);
16
17        System.out.println("Error: Updating table failed!
18        You will be redirected to the menu");
19    }
20 }

```

Listing 17: Logging in DatabaseMethods.updateTable

Hier dokumentiert `log.debug()` den Programmfluss samt übergebener Parameter. Die Platzhalter (`{}`) werden zur Laufzeit befüllt. Durch das konfigurierte Basis-Loglevel bleibt diese Meldung im Standardbetrieb jedoch verborgen.

Bei erfolgreicher Ausführung bestätigt `log.info()` die Datenbankoperation. Schlägt diese fehl, fängt ein `catch`-Block die `SQLException` ab. Der Entwickler erhält über `log.error()` detaillierte Fehlerinformationen inklusive Parameter und den `StackTrace` (`e`), während dem Nutzer parallel eine verständliche Fehlermeldung ausgegeben wird. So sind alle Logs einheitlich, informativ und klar vom User-Output getrennt.